PROJECT-BASED LEARNING

PHASE-II EVALUATION

SLOPESHIELD

Intelligent Landslide Risk & Safe Route System

Team ID: DSCPP-III-2026-T099

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T099
Semester: 3rd
Section: ML2, ML3, ML4, DS2
Domain: Disaster Management

Team Members

1. Singh, Smriti – 2510370445
2. Verma, Sanya – 2510370546
3. Verma, Jahnvi – 2510380060
4. Gaur, Anshika – 2510014858

Mentor

Mentor: Mr. Rohan Verma
Email: rohanverma.cse@geu.ac.in

Interactions: 4

Phase-I Recap & Feedback Incorporated

Phase-I

- Identify landslide risk using weather and land data.
- Find routes between selected places.
- Avoid high-risk areas when planning routes.

Phase-II Work

- Added weather API code.
- Worked on connecting the main project modules.
- Added tests for risk levels and safer routes.
- Connected route information to the map.

Key Feedback

Integrate live weather data, ensure seamless module integration, and demonstrate a functional end-to-end prototype.

Problem Statement & Finalized Objectives

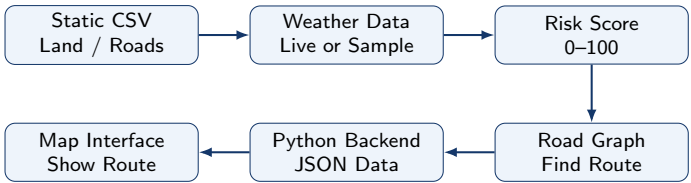
Problem Statement

Heavy rain can cause landslides and block roads in mountain areas. The shortest route may pass through dangerous places. SlopeShield helps compare routes and choose one with lower risk.

Objectives

1. Use land, road, and weather data to calculate risk.
2. Show places and roads as a graph with different travel costs.
3. Compare the shortest route with a safer route using Dijkstra's algorithm.
4. Send location and route results to the map through the backend.

Updated System Architecture / Workflow



Core Components

- Land, road, and weather data
- Risk score for each location
- Route finding and hazard testing

Phase-II Integration

- Python backend to share data
- CSV/JSON files for data
- Map showing places and routes

Module Design & Functional Requirements

Major Modules

1. Get weather data and calculate risk
2. Represent roads and find routes
3. Use classes and read/write CSV/JSON files
4. Connect the backend to the web page

Functional Requirements

- Load place and road data.
- Calculate whether each place has low or high risk.
- Compare the shortest route with a safer route.
- Show route details and test what happens when a hazard blocks a road.

Module Flow

Land + Weather Data → Risk Score → Road Graph → Find Route → Backend Data → Map

Technology Stack & Development Methodology

Technology Stack

- **Languages:** C++, Python, JavaScript
- **Frontend:** HTML, CSS, Leaflet
- **Backend:** C++ and Python
- **Data Storage:** CSV and JSON files
- **Weather API:** OpenWeatherMap
- **Tools:** VS Code, Git/GitHub

Development Methodology

1. Identify data inputs and risk factors.
2. Design risk calculation and route finding.
3. Develop each module separately.
4. Connect the modules and website.
5. Test risk levels, routes, and API requests.

Implementation of PBL Course Concepts

Course / Concept	Simple Explanation	Project Module
Data Structures & Algorithms	Represents roads as a graph and finds routes using Dijkstra's algorithm.	Route Engine
OOP with C++	Uses classes and objects to organize code and manage project data.	Risk Scoring and Backend
File Handling	Reads input from CSV files and saves results in CSV/JSON format.	Data Handlers
Networking / API	Transfers data between the backend and frontend using JSON.	`backend/server.py`

Development & Implementation Progress

Module	Status	Evidence / Limitation
OOP + File Handling	Implemented	Reads and saves data using CSV/JSON files.
Risk Scoring	Implemented	Calculates risk scores and assigns risk levels.
Graph + Routing	Implemented	Uses a graph and Dijkstra's algorithm to find routes and test safer alternatives.
Python REST API	Implemented	Provides data about locations, roads, routes, and hazard tests.
Frontend Map	Implemented	Using Leaflet and OpenStreetMap,it displays locations,risk zones and routes.
Live Weather	Partly checked	Weather API code is present, but the system mainly uses sample weather data.

Overall Progress

Core modules and automated tests are present. Live API-to-risk-to-route integration and full browser end-to-end behavior remain to be verified.

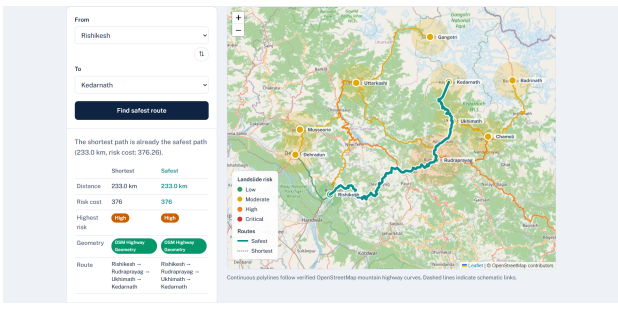
Working Prototype / Demonstration

Features to Demonstrate

- View locations and road routes.
- Compare shortest and safer routes.
- Test a hazard and possible detour.
- View route and location details.

Backend Test / Start

```
python main.py --test
python main.py --serve --port 8000
```



Testing, Results & Validation

Test	Expected Result	Result	Status
Risk Assessment	Correct risk classification for each scenario	6/6 scenarios passed	Passed
Route Engine	Correct routes using graph and Dijkstra	8/8 route scenarios passed	Passed
System Integration	Backend modules and data handlers work together	All 8 integration phases passed	Passed
Hazard Simulation	An alternate route is found during a cloudburst	Detour via UTTK and UKHI verified	Passed
REST API	Backend endpoints return valid responses	Health, route and simulation endpoints returned HTTP 200	Passed
Weather Integration	Live weather data updates risk and routes.	Live weather result not yet shown	Pending

Summary

Most tests passed. Live weather data and the complete connection between the website and backend still need to be checked.

GitHub Repository & Development Activity

Repository Details

- Project source code
- Updated README file
- Team members listed
- Latest code uploaded

Development Activity

- Code changes uploaded
- Commit history available
- Team contributions
- Project documentation

GitHub Repository:

`https://github.com/SmritiSingh118/SlopeShield.git`

Individual Contribution

Member	Role	Technical contribution	Commits
Smriti Singh	Risk Assessment	Collects and cleans data, calculates risk scores, assigns risk levels, and tests different cases	Verify
Sanya Verma	Web UI	Creates the website, adds the map, shows risk zones and routes, and handles user input	Verify
Jahnvi Verma	Route Planning	Creates the road graph, sets road weights, uses Dijkstra's algorithm to find routes, and tests route options	Verify
Anshika Gaur	OOP + Integration	Organizes code using classes and objects, handles CSV/JSON files, connects project modules, and links backend results to the website	Verify

Challenges, Remaining Work & Phase-III Roadmap

Challenges / Current Gaps

- Connecting modules made by different team members.
- Checking live weather data against sample data.
- Making sure the backend sends data the map can use.
- Adding more detailed road information.

Work Done

- Connected backend services and data files.
- Added tests for module connection and route changes.

Phase-III Roadmap

1. Check the live weather API with a valid key.
2. Confirm weather data changes the risk score and route.
3. Test the complete system in a browser.
4. Improve road data and list data sources.
5. Finish the report and prepare the project to run.

Expected Outcomes & References

Expected Outcomes

- Show risk levels for selected places.
- Compare the shortest route with a safer route.
- Test a hazard and show a possible detour.
- Use CSV/JSON data, a backend, and a map.
- Keep the project ready for future improvements.

References

1. OpenWeatherMap Current Weather API.
2. OpenWeatherMap Forecast API.
3. Leaflet: `https://leafletjs.com/`.
4. Project datasets and research papers.

Thank You!
Questions & Suggestions